
Medical Image Segmentation and Enhancement

Zhang Chi
MC451839
mc45183@um.edu.mo

Abstract

Medical image segmentation plays a pivotal role in diagnostics and treatment planning, particularly for thoracic CT scans. However, there are relatively few datasets in this area, and the Deformation Recovery Diffusion Model provides a data augmentation method to generate medical images that conform to anatomical structures. This study evaluates the performance of TotalSegmentator—a nnUNet-based automated segmentation tool—and investigates the impact of the Deformation Recovery Diffusion Model (DRDM) as a data augmentation technique.

1 Introduction

Medical image segmentation is a cornerstone of modern diagnostics and treatment planning, enabling precise localization of anatomical structures in modalities such as Computed Tomography (CT) and Magnetic Resonance Imaging (MRI).

TotalSegmentator[2] is an automatic medical image segmentation tool based on nnUNet, which aims to quickly and accurately segment the anatomy of the whole body or specific organs / tissues from medical images (such as CT or MRI). It uses the powerful performance of nnUNet and advanced deep learning technology to provide efficient and reliable segmentation results.

nnUNet[1] is a medical image segmentation method based on deep learning, which can automatically configure the whole segmentation process. It can simplify the complexity of medical image segmentation tasks, and make it easy for non expert users to use while maintaining high performance. I choose nnUNet for the following reasons: (1) nnunet has high performance and low demand for computing resources. (2) nnunet can adapt to different datasets, simplifying the process of building datasets. (3) totalsegmentator is developed based on nnunet, which can be compared with

DRDM (Deformation Recovery Diffusion Model) [3] is a diffusion model-based generative model that focuses on generating diverse and anatomically reasonable image deformations through deformation fields, rather than directly generating images. DRDM can increase the diversity of the dataset and improve the generalization ability of the model in the case of limited data volume. And unlike traditional image enhancement techniques, DRDM can maintain a reasonable anatomical structure and avoid generating unreasonable deformations

This paper will discuss the performance of totalsegmentator and basic nnUNet through a series of experiments, and the impact of data enhancement effect of Deformation Recovery Diffusion Model (DRDM) on nnUNet training

2 Dataset construction and preprocessing

2.1 Dataset construction

The data used in this article are all from the CT and MRI datasets of the totalsegmentator <https://github.com/wasserth/TotalSegmentator?tab=readme-ov-file>. The CT dataset

can segment 117 important anatomical structures, and the MRI dataset can segment 50 anatomical regions. The CT dataset directory is as follows:

```
CT_dataset/  
├── s0029/  
│   ├── ct.nii.gz  
│   └── segmentations/  
│       ├── adrenal_gland_left.nii.gz  
│       ├── adrenal_gland_right.nii.gz  
│       └── ...  
└── ...
```

The segmentation folder contains the real segmentation results of 117 organs or parts. And the structure of the MRI dataset is similar to this.

Due to the large size of the totalsegmentator dataset and the optimization of DRDM for cardiac MRI and pulmonary CT, I need to create a subset specifically for cardiac MRI or pulmonary CT. In practical operation, the raw data was input into a 3D slicer for viewing. As long as the heart or lungs appear mostly or entirely in the image, and the whole body image is excluded, the image would be drawn into subsets. However, there were too many cardiac MRI images from the totalsegmentator, so I ultimately only chose to construct a subset of lung CT scans. There are 51 cases in this subset of data.

2.2 Data preprocessing

The CT data quality of the total segmenter is very high, and when viewed in a 3D slicer, there is almost no noise (Figure.1). In practical operation, I have tried various denoising operations, but the results have all reduced the accuracy of the final segmentation. I found that the contrast of the original data was low during observation, and some organs had similar colors in the original data. Therefore, I tried methods such as histogram averaging to improve image contrast, but the results also reduced the accuracy of the final segmentation. So I think there is no need for preprocessing operations on the CT dataset of the total segmenter, as preprocessing operations can actually reduce the final segmentation effect.

However, the MRI data of the total segmenter is different from the CT data. When I observed the 3D slicer, I found that most MRI data had a lot of noise similar to salt and pepper noise(Figure.2), and there were also cases of low contrast. So I first used the method of increasing contrast, which also resulted in a decrease in the accuracy of the final segmentation. Then I used Gaussian filtering to remove noise. However, Gaussian filtering can damage the edge information of the image and may result in blurred boundaries. So before using Gaussian filtering, I first perform median filtering on the original data.

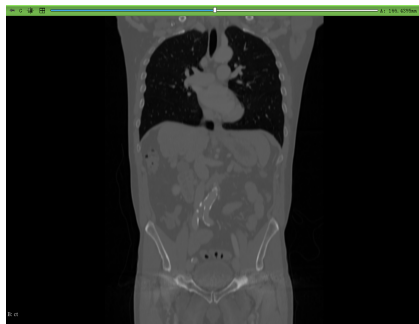


Figure 1: Original CT data



Figure 2: Original MRI data

In tasks such as medical image segmentation, Dice Score, Jaccard Index, and Hausdorff Distance are core indicators for evaluating the consistency between segmentation results and real labels.

(1)**Dice score** measures the degree of overlap between the predicted segmentation results and the actual labels. The closer Dice is to 1, the higher the overlap between the segmentation result and the

Data	Dice Score	Jaccard Index (IoU)	Hausdorff Distance
Original_MRI	0.9470	0.8994	10.1980
Only Gauss	0.9471	0.8995	7.6158
Gauss + Median	0.9333	0.8750	6.7082

Table 1: The influence of different filtering methods on segmentation results

real label.

$$\text{Dice} = \frac{2 \cdot |A \cap B|}{|A| + |B|} \quad (1)$$

(2)**Jaccard index (IoU)** also measures the degree of regional overlap, and similarly, the closer the Jaccard index is to 1, the more accurate the segmentation.

$$\text{Jaccard} = \frac{|A \cap B|}{|A \cup B|} \quad (2)$$

The mathematical relationship between Jaccard index and Dice score is as follows:

$$\text{Jaccard} = \frac{\text{Dice}}{2 - \text{Dice}} \quad (3)$$

(3)**Hausdorff distance** measures the maximum spatial mismatch between the predicted segmentation boundary and the true label boundary. The smaller this value, the closer the segmentation boundary is to the true boundary

$$H(A, B) = \max \left\{ \sup_{a \in A} \inf_{b \in B} d(a, b), \sup_{b \in B} \inf_{a \in A} d(a, b) \right\} \quad (4)$$

I have chosen the MRI dataset with the case name "s0029" from the totalsegmentator as an example for display. The impact of using only Gaussian filtering or a mixture of Gaussian filtering + median filtering on the final segmentation results of the original data is shown in the Table.1. The segmentation result here is that I input the filtered data into the totalsegmentator for segmentation processing, and select the heart part for comparison.

The results show that the data after Gaussian filtering can effectively reduce the Hausdorff distance in subsequent segmentation. Compared to using only Gaussian filtering, hybrid filtering significantly reduces the Hausdorff distance (about 10%) while slightly reducing the Dice and Jaccard values (about 1%)

3 TotalSegmentor Benchmarking

The installation tutorial for totalsegmentator follows the official GitHub repository: <https://github.com/wasserth/TotalSegmentator>. The main issue encountered when deploying the total segmenter locally is conflicts between numpy and other libraries. Finally I successfully deployed the totalsegmentator on my Windows10 laptop, with Python=3.9, pytorch=2.3.0.

CPU = Intel Core i5-9300H

GPU = Nvidia Quadro P600

For CT data: Run TotalSegmentator -i path/to/ct.nii.gz -o path/to/segmentations

For MRI data: Run TotalSegmentator -i path/to/mri.nii.gz -o path/to/segmentations --task total_mr
When I segment a single CT image locally(using --fast), it takes a long time (about 5 minutes). So sometimes, in order to quickly obtain results, I use the official online platform provided by Totalsegmenter: <https://totalsegmentator.com/> Running the above command can obtain independent segmentation results for all parts, rather than merging all results into one file. Using --ml can save the final segmentation results in a nifti file.

4 nnUNet Benchmarking

Due to the low computing power of my laptop, I initially attempted to deploy nnUNet in the colab provided by Professor Li. But Colab keeps reminding a library that it needs to restart the runtime to

use it, but no matter how many times the runtime is restarted, the same prompt will appear. After resolving this issue, Colab was granted permission to edit a JSON file without authorization. The JSON file was uploaded by me to Colab. The original file permissions are readable and writable, but Colab cannot read the file and cannot modify permissions. So I successfully deployed nnUNet using the computer E11-1046 from UM. The computer configuration is as follows: python = 3.9 pytorch = 2.6.0

CPU = Intel Core i7-13700

GPU = Nvidia RTX 4090 24GB

The installation tutorial for nnUNet follows the official GitHub repository: https://github.com/MIC-DKFZ/nnUNet/blob/master/documentation/installation_instructions.md. When I deploying nnUNet on my own laptop, I also encountered conflicts between numpy and other libraries. When I deployed nnUNet in the school computer room, because the computer did not have pip and it was not allowed to install pip without permission, I needed to install all the required packages using conda command.

I used 45 of the 51 CT data from the total segmenter as the training set and the remaining 6 as the validation set. nnUNet uses five fold cross validation, which divides the dataset equally into five small subsets, and then runs modeling on different subsets to obtain indicators of the fitting effect of each subset model. This method can fully utilize the small dataset, but it is time-consuming. In actual experiments, each fold takes 3 to 4 hours (while epoch=100). I observed that the training results of each fold were similar, so I chose the third fold and extended the training time appropriately (epoch=150, and it took 5 to 6 hours), resulting in the following training results (Figure,3).



Figure 3: nnUNet training result fold=2 epoch=100

From the results, it can be seen that the final trained Dice is about 0.65, which is lower than the totalsegmentator. I think the reasons are as follows:

- (1). The real segmentation results obtained by the totalsegmentator contain too many irrelevant parts. In the CT dataset I created, which focuses on the lungs, the vast majority of the data includes not only the lungs, such as abdomen, neck and even head. However, most of the 117 labels did not appear, so nnUNet may have incorrectly learned other parts or noise.
- (2). The nnUNet network training only allows label files to be input as a single file, meaning that each raw data corresponds to a label file that contains all segmented organs or parts. So I need to merge the real segmentation results provided by the total segmenter into one label file. I compared the merged label file with the original CT image, and the Dice score was approximately 0.63. There may be errors between the real segmentation results and the original image, and the accumulated

errors continue to affect the training of nnUNet’s network, resulting in poor final training results. (3). Although the experimental results show that dice converges approximately around 150epoch, and the rise of dice slows down around 140epoch, it may achieve better convergence after longer training.

5 DRDM results

The installation tutorial for DRDM follows the official GitHub repository: https://github.com/jianqingzheng/def_diff_rec. I successfully deployed DRDM according to GitHub content, and input 45 pairs of original image and label files into DRDM. Only one interpolation method is allowed in the configuration file of the model.

In the first processing, I chose the default linear interpolation method of DRDM, and obtained a fuzzy label file, which cannot be used as label training.

In the second processing, I changed the interpolation method to the nearest neighbor interpolation method (because this interpolation method can protect the edge information and reduce ambiguity), and obtained the results shown in the Figure.4. The contents of the three windows have changed, that

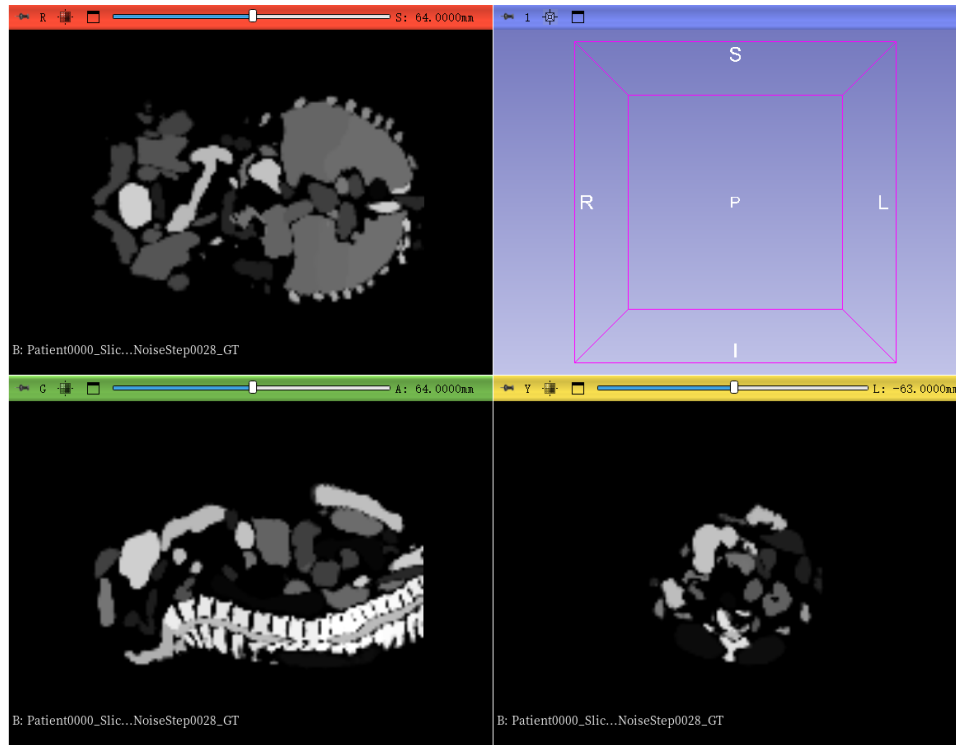


Figure 4: DRDM processing result

is, the horizontal slice view has changed to the front view, the malformed slice view has changed to the top view, and the coronal slice view has changed to the side view, and the body has changed from standing to lying. This means that the image has been rotated.

An the results showed that the label file had strange structures (Figure.5). The image shows that the label file enhanced by DRDM exhibits unreasonable body structures, such as large holes in the lungs, fragmented heart, and strange intestines in the abdomen. The holes or gaps in the lungs can be understood as DRDM producing the diseased lungs. But the originally unified heart has split into several scattered parts, and the intestines have become block like, which are difficult to understand.

Due to the failure of DRDM enhancement, I am also unable to train nnUNet on the enhanced dataset.

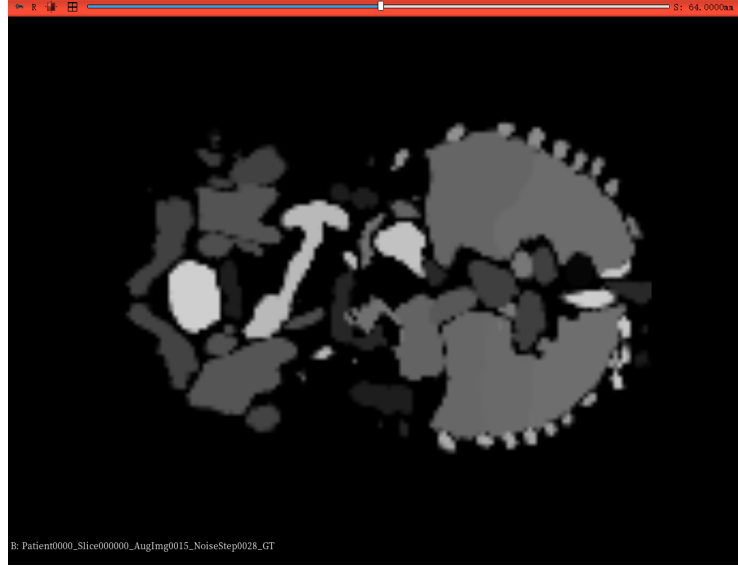


Figure 5: Wrong structure in results of DRDM

6 Analysis and Results

The biggest challenge in this project is that myself computer’s computing power is too low to support network training. I can only use the school’s computer for network training, and the same computer may be used by different people. Because my data and environment are stored on the same computer, sometimes I need to wait for others to work on the computer before I can conduct network training. This also increases time consumption.

The nnUNet training took three to four days, and the DRDM processing took two to three days.

The totalsegmentator is developed based on nnUNet, so the segmentation effect should be better than nnUNet, and the experimental results have also verified this point. Moreover, the paper[2] also mentions that the training of the totalsegmentator requires less than 12GB of RAM and does not require GPU to train. However, nnUNet has a high demand for computing resources, and training with 4090 GPU still requires an extremely long time.

Traditional image enhancement methods may produce unreasonable anatomical structures, while DRDM can generate structures that conform to anatomy. However, the computational resources required for DRDM enhancement are also high, and it takes a long time to run on 4090 GPU.

References

- [1] Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. nnu-net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2):203–211, 2 2021.
- [2] Jakob Wasserthal, Hanns-Christian Breit, Manfred T. Meyer, Maurice Pradella, Daniel Hinck, Alexander W. Sauter, Tobias Heye, Daniel T. Boll, Joshy Cyriac, Shan Yang, Michael Bach, and Martin Segeroth. Totalsegmentator: Robust segmentation of 104 anatomic structures in ct images. *Radiology: Artificial Intelligence*, 5(5):e230024, 2023.
- [3] Jian-Qing Zheng, Yuanhan Mo, Yang Sun, Jiahua Li, Fuping Wu, Ziyang Wang, Tonia Vincent, and Bartłomiej W. Papiez. Deformation-recovery diffusion model (drdm): Instance deformation for image manipulation and synthesis. *CoRR*, abs/2407.07295, 2024.